

Milestone 5 Report

Smart MCQ Solver Challenge

Course: BSDA2001P – Introduction to Deep Learning and

Generative AI Name: Soumyadip Hazari

Student ID: 24f3004946

Objective

The primary objective of this milestone is to improve the prediction accuracy and the performance of the Smart MCQ Solver by implementing inference optimization and ensemble learning techniques. The milestone involved performing inference using fine tuned DeBERTa and RoBERTa sequence classification models, converting model logits into probability distributions using the softmax function and extracting the top 3 ranked answers options. Additionally, TTA is introduced by generating multiple prompt variations during inference and averaging the resulting predictions to improve models robustness.

Work Performed

1. Import Required Libraries and Loading Dataset—

The first step of the milestone contains the importing necessary libraries required for the data manipulation, transformer models loading, GPU-based inference, probability computation and ensemble prediction. For this purpose, pandas, numpy torch, torch.nn.functional , hugging face transformers and tqdm were imported.

Later By using pandas we loaded train.csv, test.csv and sample_submission.csv and also created a copy of each datasets to make sure that original datasets never get changed

2. Label Mapping

A label mapping dictionary was created to map the numerical output labels produced by the transformer models to their corresponding MCQ answer options(A,B,C,D and E). This mapping was later used to convert model predictions into the required competition format while generating top-1 and top-3 predictions.

```
id_label = {  
    0: "A",  
    1: "B",  
    2: "C",
```

```
3: "D",  
4: "E",  
}
```

Observations :

- Established a bidirectional mapping between numerical labels and options.
- Simplified prediction interpretation and submission generation

3. Loading Transformer Models

DeBERTa and RoBERTa sequence classification models along with their respective tokenizers were loaded using the hugging face transformers library.

```
DEBERTA_MODEL = "microsoft/deberta-v3-small"  
ROBERTA_MODEL = "roberta-base"  
  
deberta_tokenizer = AutoTokenizer.from_pretrained(DEBERTA_MODEL)  
roberta_tokenizer = AutoTokenizer.from_pretrained(ROBERTA_MODEL)  
  
deberta_model = AutoModelForSequenceClassification.from_pretrained(  
    DEBERTA_MODEL,  
    num_labels=5  
) .to(device)  
  
roberta_model = AutoModelForSequenceClassification.from_pretrained(  
    ROBERTA_MODEL,  
    num_labels=5  
) .to(device)  
  
deberta_model.eval()  
roberta_model.eval()
```

Then the models were configured with five output labels corresponding to five answer options and transferred to the GPU for efficient inference.

4. Softmax Probability Computation

A helper function was implemented to cover the raw logits produced by the transformer models into probability distributions using softmax function. These probabilities were later used for ranking answer options and performing ensemble operations.

```
# Softmax helper
def get_probabilities(logits):
    return F.softmax(logits, dim=-1)
```

5. Input Formatting and Prediction Utilities

Multiple helper functions were implemented to simplify inference. The prompt and answer choices were concatenated into required input format, function was implemented to combine predictions from both transformer models.

```
# MCQ Formatting function
def build_inputs(row):
    prompt = str(row["prompt"])

    choices = [
        str(row["A"]),
        str(row["B"]),
        str(row["C"]),
        str(row["D"]),
        str(row["E"]),
    ]

    return [prompt + " [SEP] " + choice for choice in choices]
```

Formats each multiple-choice question by combining the question prompt with each of the five answer options using the [SEP] separator.

```
# Top-3 prediction helper
def top3_predictions(probabilities):
    indices = np.argsort(probabilities)[:,-1][:3]
    labels = [id_label[i] for i in indices]
    return " ".join(labels)
```

The function sorts the probability values in descending order, selects the indices of the top three probabilities, converts them into their corresponding answer labels (A–E), and returns the prediction in the Kaggle submission format (e.g., A C D).

```
# Weighted Ensemble helper
def weighted_ensemble(
    deberta_probs,
    roberta_probs,
    w_deberta=0.7,
    w_roberta=0.3,
):
    return (
        w_deberta * deberta_probs
        + w_roberta * roberta_probs
    )
```

The function multiplies the DeBERTa probabilities by 0.7 and the RoBERTa probabilities by 0.3, then adds them together to obtain the final probability distribution. This ensemble approach gives greater importance to the DeBERTa model while still incorporating the predictions from RoBERTa.

Worked done –

- Standardized MCQ input formatting
- Automated Top-3 prediction generation.
- Implemented reusable ensemble prediction function

6. Single Model Inference

Inference was performed independently using the DeBERTa and RoBERTa models. For a selected sample, the predicted probabilities of all answer options were computed and the option with the highest probabilities was identified. This served as the baseline before applying ensemble techniques.

```
@torch.no_grad()
def predict_probabilities(model, tokenizer, row, max_length=256):

    inputs = build_inputs(row)

    probs = []

    for text in inputs:
```

```

encoding = tokenizer(
    text,
    return_tensors="pt",
    truncation=True,
    padding="max_length",
    max_length=max_length,
)

encoding = {k: v.to(device) for k, v in encoding.items()}

logits = model(**encoding).logits

probability = get_probabilities(logits).squeeze()

if probability.numel() == 5:
    probs.append(probability.max().item())
else:
    probs.append(probability[-1].item())

probs = np.array(probs)
probs = probs / probs.sum()

return probs

```

The first function `build_inputs` formats the MCQ into five prompt-option input pairs. Each input is then tokenized and passed through the transformer model to obtain logits. These logits are converted into probabilities using the Softmax function, and the probability corresponding to each option is collected. Finally, the probabilities are normalized so that their sum equals 1, resulting in a valid probability distribution across all five answer choices.

7. Simple and Weighted Probability Ensembling

The probability distributions generated by both models were combined using two ensemble strategies. First, simple probability averaging was performed by equally averaging the outputs of both models. Next, weighted probability averaging was implemented by assigning weights of **0.7** to DeBERTa and **0.3** to RoBERTa. The resulting probabilities were ranked to generate the final Top-3 predictions.

I. Simple Probability Averaging (Question 2)

```
avg_probs = (deberta_probs + roberta_probs) / 2

print("Averaged Probabilities")

for i, p in enumerate(avg_probs):
    print(f"{id_label[i]} : {p:.6f}")

best_idx = np.argmax(avg_probs)

print("\nHighest Averaged Prediction:")
print(f"{id_label[best_idx]}, {avg_probs[best_idx]:.6f}")
```

The implementations are – $P_{avg} = (P_{Deberta} + P_{RoBERTa})/2$

Where we tried to find the answer of the “Which answer option receives the highest averaged probability after simple probability ensembling?”

- II. And later in question 3 we tried to find the answer of “Which answer option is ranked first after weighted `ensembling`?”

Where we implemented

```
def weighted_ensemble(
    deberta_probs,
    roberta_probs,
    w_deberta=0.7,
    w_roberta=0.3,
```

```

):
    return (
        w_deberta * deberta_probs
        + w_roberta * roberta_probs
    )

weighted_probs = weighted_ensemble(
    deberta_probs,
    roberta_probs,
    w_deberta=0.7,
    w_roberta=0.3
)

print("Weighted Ensemble Probabilities")

for i, p in enumerate(weighted_probs):
    print(f"{id_label[i]} : {p:.6f}")

best_idx = np.argmax(weighted_probs)

print("\nHighest Weighted Prediction:")
print(f"{id_label[best_idx]}, {weighted_probs[best_idx]:.6f}")

```

Where we implemented $P_{final} = 0.7 \times P_{DeBERTa} + 0.3 \times P_{RoBERTa}$

8. Test - Time Augmentation

Test-Time Augmentation was implemented by creating two versions of every prompt: the original prompt and an instruction-augmented prompt. Predictions from both inference passes were averaged to obtain a more robust probability distribution, and the effect of TTA on Top-1 predictions was analyzed.

In Question 6 we tried to find out “How many of the first 50 rows produce a different Top-1 prediction after applying Test-Time Augmentation?”

```
def build_inputs_augmented(row):  
    instruction = "Answer the following multiple-choice question  
carefully: "  
  
    prompt = instruction + str(row["prompt"])  
  
    choices = [  
        str(row["A"]),  
        str(row["B"]),  
        str(row["C"]),  
        str(row["D"]),  
        str(row["E"]),  
    ]  
  
    return [prompt + " [SEP] " + choice for choice in choices]
```

Here we created the augmented prompt. Then we applied inference on both versions

```
for _, row in test_cc.head(50).iterrows():  
  
    # Original prompt  
    original_inputs = build_inputs(row)  
    original_probs = predict_from_inputs(  
        deberta_model,  
        deberta_tokenizer,  
        original_inputs  
    )
```

```

augmented_inputs = build_inputs_augmented(row)

augmented_probs = predict_from_inputs(
    deberta_model,
    deberta_tokenizer,
    augmented_inputs
)

```

Then we found the average for the both predictions

```

tta_probs = (original_probs + augmented_probs) / 2

```

And later we computed the top-1 prediction

```

original_top1 = np.argmax(original_probs)
tta_top1 = np.argmax(tta_probs)

if original_top1 != tta_top1:
    changed += 1

print("Number of changed Top-1 predictions:", changed)

```

Key Results

Experiment	Result
DeBERTa Highest Prediction	A (0.202426)
Simple Ensemble Highest Prediction	A
Weighted Ensemble Highest Prediction	A
Top-3 Prediction String	A C D
Submission Prediction Rows	500
TTA Changed Top-1 Predictions	21
Different Top-1 after Ensembling	7
Positive Confidence Gain	0
Changed Top-3 Rankings	19
Final MAP@3 Score	0.2817